ChatGPT

Claude Code – Workflows & Config (Contradictions Resolved as of Jan 2026)

Contradictions and Resolutions

Topic	**Claim A** (Earlier Source)	**Claim B** (Updated Source)	**Resolution (Jan 2026)**	Confidence	Sources
Plan vs. "Think" Mode Shortcut	*Claim A:* Hitting <kbd>Tab</kbd> was suggested as a way to enter a planning or thinking mode (not clearly distinguishing plan vs extended thinking)	*Claim B:* Plan Mode is activated via <kbd>Shift+Tab</kbd> (twice in sequence) or the `/plan` command, whereas <kbd>Tab</kbd> alone toggles **Extended Thinking** (more reasoning) [1] [2]	**Plan Mode** requires an explicit trigger (double <kbd>Shift+Tab</kbd> or `/plan`), distinct from **Extended Thinking** (single <kbd>Tab</kbd> for deeper reasoning) [1] [2]. These are separate features with different purposes.	1.0	[Claude official docs, community guides] [1] [2]

Topic	**Claim A** (Earlier Source)	**Claim B** (Updated Source)	**Resolution (Jan 2026)**	Confidence	Sources
Subagent Tool Restrictions	*Claim A:* Each subagent needed a strict **allowlist** of every permitted tool for safety (implying tedious manual listing of allowedTools per agent) [3]	*Claim B:* Subagents support a **deny-list** (`disallowedTools`) field – introduced in v2.0.31+ – allowing a "negative" security model to block only unsafe tools and implicitly allow the rest [4] [3]	**Deny-list model is supported.** Instead of allow-listing all safe tools, you can specify `disallowedTools` in the agent's YAML/MD definition to block certain tools (e.g. deny Edit/Write/Bash) while permitting all others [4] [3]. This is the recommended approach in Claude Code v2.0.31+ (safer and easier to maintain).	1.0	[3] [4]
Hooks vs. Subagents for Policy	*Claim A:* Emphasis on using **subagents** (separate Claude instances with restricted tools) to enforce code review and safety policies.	*Claim B:* Use **hooks** (deterministic scripts) for hard enforcement (e.g. blocking secrets or dangerous commands), and reserve subagents for higher-level analysis or subjective checks [5].	**Split strategy:** Use **PreToolUse hooks** for strict, binary rules (they can `exit 2` to deny an operation deterministically) and use **subagents** for complex or subjective tasks (code quality review, audits) where an AI's judgment is needed [5] [6]. Hooks and subagents serve different roles in an optimized workflow.	0.9	[5] [6]

Topic	**Claim A** (Earlier Source)	**Claim B** (Updated Source)	**Resolution (Jan 2026)**	Confidence	Sources
Chrome Integration Availability	*Claim A:* Chrome browser automation is a feature for Claude Pro/Max tiers, with **"no WSL support"** noted (implying it may only work on native macOS/Linux) [7].	*Claim B:* Chrome integration **does** require Claude Pro/Max+ and a Chrome extension, and indeed only works on native Windows/macOS (not under WSL); this OS requirement was confirmed explicitly [8].	**Pro/Max feature on native OS only.** Claude Code's `/chrome` automation is available to Pro/Max subscribers with the official Chrome extension installed. It does **not** function in WSL environments (must run in a normal Windows or macOS setup) [7] [8]. This is now clearly documented, preventing confusion.	1.0	[7] [8]
Plugin Installation Mechanism	*Claim A:* Early material mentioned **plugins** in Claude Code but provided no clear distribution method (implying plugins might not be easily shareable or had to be manual).	*Claim B:* Claude Code has an **official plugin marketplace** system: plugins are packaged as GitHub repos and installed via `/plugin marketplace add <username>/<repo>` given a proper manifest (`.claude-plugin/marketplace.json`) [9]. Also, an Anthropic-curated repo (`claude-plugins-official`) allows one-click install of approved plugins [10].	**Marketplace-based plugins.** Claude Code supports shareable plugins distributed as GitHub repositories. Each plugin repo needs a manifest (JSON) listing its commands/skills/hooks. Users install them with `/plugin marketplace add user/repo` or via the official `claude-plugins-official` registry for vetted plugins [9] [10]. Plugins are **not** installed via NPM; they are fetched as repo bundles.	0.95	[9] [10]

Topic	**Claim A** (Earlier Source)	**Claim B** (Updated Source)	**Resolution (Jan 2026)**	Confidence	Sources
MCP Configuration Scope	*Claim A:* MCP (Model Context Protocol) server configs were thought to apply globally (no project isolation) – i.e. adding an external tool server would affect all projects, with configuration in a single user config file.	*Claim B:* Claude Code supports scoping MCP servers per **user**, **project**, or **local directory**. By default, `claude mcp add` adds a server config for the current directory (or project) unless `--scope user` is specified. Global configs live in the user's `~/.claude/claude.json` (not in each project), while project-specific servers are stored in a project's `.mcp.json` file [11] [12].	**Scoped MCP configs.** By default, MCP servers you add are local to the project/directory. Use `--scope user` to register a tool globally (persisted in `~/.claude/claude.json` under `mcpServers`) [13] [14]. Project-scoped servers are saved to a `.mcp.json` in that project so they don't pollute others [15]. This hierarchy prevents unwanted tool access outside intended scope.	0.9	[11] [12]

Corrected Reference Sheet (Claude Code Config & Workflow, Jan 2026)

Core Commands & Shortcuts

- **Mode Switching**: Claude Code has multiple permission modes. By default it **asks** on first tool use. You can cycle modes in-session with <kbd>Shift+Tab</kbd>: *Default* → *acceptEdits* → *Plan*. Hitting <kbd>Shift+Tab</kbd> twice enters **Plan Mode**, where Claude only analyzes/plans but does **not** execute edits until you approve [1] [2]. (Approving a plan will drop back into acceptEdits mode automatically, unless you disable it [2].)
- **Extended Thinking**: Pressing <kbd>Tab</kbd> (in default mode) toggles **Extended Thinking** mode, which makes Claude spend more tokens on reasoning before responding [16]. Use this for complex refactoring or algorithm design – it slows responses and increases cost, so avoid for trivial tasks [16].
- **Background Tasks**: Use **Ctrl+B** to send a running command to the background (freeing Claude to continue with other tasks) [16] [17]. Press Ctrl+B twice if using a tmux session [17]. The background process's output can later be retrieved via the `TaskOutput` tool.
- **Rewind**: Press **Esc** twice to trigger a **Rewind**, which undoes the last conversation turn *and* reverts code state changes from that turn [18]. *(Note: Rewind may not catch every filesystem change made, so use with caution)* [18].

- **Other Useful Shortcuts**: `@` in the chat triggers file path autocompletion (inserts an @-mention reference to a file) [19] . The <kbd>#</kbd> key (in chat) lets you quickly append content to **CLAUDE.md** memory mid-session [20] . If Vim mode is enabled (`/vim` command), standard **Vim keys** (h/j/k/l, etc.) work for navigation in the editor [21] .

Config File Locations & Precedence

- **User Settings**: `~/.claude/settings.json` – user-wide defaults applied to all projects [22] . Define overall preferences here (defaultMode, global allow/deny patterns, etc.). This is the lowest precedence in merge order.
- **Project Settings**: `<project>/.claude/settings.json` – team or repo-specific settings (checked into version control to share defaults with your team) [22] . Claude loads this on entering the project directory.
- **Local Overrides**: `<project>/.claude/settings.local.json` – personal overrides for the project (not committed to git) [22] . Use this for machine-specific or sensitive config (API keys, personal tool preferences).
- **Enterprise Policy**: **managed-settings.json** – an IT-managed config that enforces organization-wide settings. On macOS this lives at `/Library/Application Support/ClaudeCode/managed-settings.json` (on Linux/WSL: `/etc/claude-code/managed-settings.json`) [23] . If present, it overrides all other settings (highest precedence) [23] . *This file is typically used only in enterprise deployments.*
- **Precedence**: The above settings are merged such that **enterprise** > **local project** > **project** > **user** (first match wins for any specific key) [24] [22] . In practice, managed-settings can disable or force certain configs across all users, while individuals can still have per-project tweaks that override their global user defaults.

Schema Formats (Skills, Agents, Hooks, Settings)

- **Skill Files**: Located in `~/.claude/skills/` (for global skills) or `.claude/skills/` in a project (for project-specific skills) [25] . Each skill is a Markdown file (often named `SKILL_NAME.md`) with a YAML frontmatter. The frontmatter includes at minimum a `name` and a `description` of the expertise. The **description** is critical – Claude uses it to auto-load relevant skills (matching the context) [26] . The content after the frontmatter can include instructions, examples, or knowledge that Claude will inject when relevant [26] . *(Typical use: a SECURITY.md skill file providing secure coding guidelines that Claude applies during code generation.)*
- *Example Skill frontmatter:*

```
---
name: security-audit
description: Static analysis and security best practices for web apps
(OWASP top 10)
---
```

  This would auto-activate whenever the context suggests a security check is needed. Skills consume ~30-50 tokens each time they run, so keep them focused [27] .
- **Subagent Definition (Agent)**: Custom subagents are defined as Markdown files in `~/.claude/agents/` or `.claude/agents/` (project) [28] . The file uses a YAML frontmatter to specify the

subagent's **name**, **description**, allowed **tools**, and optionally its `model` and permission overrides. By default, an agent's `tools` list is an allow-list – it will only have those tools available. In Claude Code v2.0.31+, you can also specify a `disallowedTools` list to *deny-list* certain tools [4] [3]. If both are present, the agent effectively has all tools except those in disallowed list (since unspecified safe tools are allowed).

- *Example Agent frontmatter:*

```
---
name: researcher
description: Analyzes code and docs (read-only assistant)
tools:
  - Read
  - Grep
  - Glob
  - LS
disallowedTools:
  - Edit
  - Write
  - Bash
model: sonnet
---
```

This defines a **"Researcher"** agent that can read and search, but cannot modify files or run shell commands [29] [30]. Agents are invoked via the `/agents` menu or programmatically via the Agent SDK, and each runs isolated from the main session [31].

- **Hooks**: Hooks are configured in the **settings.json** (any level) under a `"hooks"` object. Claude Code supports hooks at various lifecycle events: e.g. `PreToolUse`, `PostToolUse`, `PostPlan`, `Stop` etc. [32]. Each hook entry can have a **matcher** (to filter when it runs, based on tool name or file path pattern), a shell **command** to execute, and a `blocking` flag [32] [33]. If a PreToolUse hook's command exits with code `2`, Claude will **deny** the tool action (as if user said "No") [32] [34]. Exit code `0` allows, and `1` signals an error. This mechanism lets you enforce policies (for example, block any `Write` to `.env` files by matching on tool=Write and path `.env` [35]).
- *Hook example:* Adding to settings:

```
"hooks": {
  "PreToolUse": {
    "matcher": { "tool": "Write", "input": { "path": ".env" } },
    "command": "exit 2",
    "blocking": true
  }
}
```

This hook will intercept any file write to an `.env` file and exit 2, causing Claude to immediately deny the action [32] [34]. Use hooks for automation too (e.g. a PostToolUse hook to run `prettier --write` on files after edits). Complex logic can be moved into external scripts (especially for regex or multi-step checks) – your hook can just call a script file in `.claude/hooks/` [36] [37].

- **Settings Schema**: The settings JSON supports fields for permission rules and various behaviors. Key fields include:
- `permissions.allow` / `permissions.ask` / `permissions.deny`: Arrays of rule patterns that override prompt behavior [38]. These support glob syntax for Bash commands and exact matches for other tools. **Order of evaluation** is always Deny → Ask → Allow (first match applies) [39]. For example, you might put `"Read(./secrets/**)"` in deny to block reading any secrets folder [40], or `"Bash(git push *)"` in ask to always confirm pushes.
- `defaultMode`: sets the starting permission mode (e.g. `"plan"`, `"acceptEdits"`, etc.) when Claude launches [41] [42].
- `alwaysThinkingEnabled`: if true, Claude will default to Extended Thinking mode for all sessions (off by default) [43].
- `sandbox.enabled`: enable the Docker-based sandbox for running Bash commands in isolation (false by default) [44]. There are related sandbox configs like `excludedCommands` to always run outside sandbox (e.g. keep `git` unsandboxed) [45].
- Many other tunables exist (output styles, plan storage path, spinner options, etc.) – see official docs for full schema [46] [47].

Permission Model and Modes

- **Permission Hierarchy**: Claude Code's *safety prompt system* uses a hierarchy of rules. When Claude attempts a tool action, it checks your permission config in this order: **Deny rules first, then Ask rules, then Allow rules** [39]. The first matching rule determines what happens [39]. Thus, a deny pattern will always override an allow pattern for the same action. For example, if `"WebFetch"` is in deny, Claude won't ever hit an allow rule for WebFetch. This hierarchy ensures explicit denials take precedence for safety.
- **Permission Modes**: There are four modes that govern Claude's prompting behavior [48]:
- **default** – Standard mode (ask on first use of each tool, then remember choice).
- **acceptEdits** – Auto-approve file edits (no prompt for Edit/Write tools), but still ask for risky ops like Bash or web fetch. Great for trusted projects to avoid repetitive "Allow edit?" prompts [49] [50].
- **plan** – Analysis-only mode. Claude will not make file changes or execute commands at all; it will only propose a plan or analysis. Use this for code reviews or exploratory discussions [51] [52]. *(In plan mode, you must manually approve any steps to actually execute them.)*
- **bypassPermissions** – No prompts *at all*. Claude will execute any tool it deems necessary without asking [53] [54]. **Use with caution:** this requires a safe sandboxed environment or non-critical context, as it literally bypasses all safety checks. It's mostly intended for CI pipelines or other controlled settings [55]. (This mode is activated by the `--dangerously-skip-permissions` flag or an enterprise policy; it may be disabled by admins via config) [56].
  You can set the default startup mode via the `"defaultMode"` setting or change modes on the fly with the <kbd>Shift+Tab</kbd> shortcut or `/mode <name>` command. For instance, to quickly allow all file edits in a session, you might switch to acceptEdits mode.

Headless Mode & Output Formats

- **Interactive vs. Headless**: Running `claude` with no flags launches the interactive REPL UI. For **non-interactive** use (scripts, CI), you can run Claude Code in one-shot mode using `claude -p "<query>"`. This executes a single prompt or command and then exits [57].
- **Output Formats**: In headless mode, use the `--output-format` flag to control how Claude's response is formatted. The default is a human-readable text, but for automation you can get

structured output. For example, `--output-format stream-json` returns a JSON Lines stream of assistant messages [57]. Each line is a JSON object containing the content (and metadata) of Claude's answer, which is ideal for parsing by other programs [58]. There's also a plain `--output-format json` (for a single JSON array of messages), and formats like `diff` or `yaml` for certain contexts. In CI pipelines, **stream-json** is commonly used to feed Claude's output into dashboards or subsequent processing [58].

- **No Interactivity in Headless**: Note that in `-p` mode, Claude will not pause to ask clarifying questions – it has no user to confirm with. It will do its best with the given prompt and tools available, then terminate. Ensure you provide all necessary context upfront since you can't refine the prompt mid-run [59].
- **Use Cases**: Headless mode is perfect for automating code analysis on multiple files (e.g., loop over files and run `claude -p "Analyze $file" --output-format stream-json` for each) [58]. It's also used in GitHub Actions via the `anthropics/claude-code-action` (which effectively triggers Claude headlessly to handle PRs or issues) [60].

Plugin Installation & Marketplace

- **Claude Plugins**: Plugins in Claude Code are bundles of commands, skills, hooks, agents, and/or configurations that can be shared and reused. They allow teams or the community to distribute workflow enhancements easily [61]. Each plugin is essentially a GitHub repository containing Claude config files.
- **Marketplace Manifest**: A plugin repo **must include** a manifest file for Claude to recognize it. In the official marketplace system, this file is named `marketplace.json` and is placed in a `.claude-plugin/` directory at the root of the repo [9]. (In other contexts, it may be called `plugin.json` – the key is that Claude looks for a manifest in the plugin folder) [62]. The manifest lists the plugin's name, version, description, and which files to include (e.g. which commands, skills, agents from that repo) [63] [64]. Claude will load those assets when the plugin is installed.
- **Installing Plugins**: There are two ways to install plugins:
- **GitHub Marketplace** – If the plugin is public on GitHub, use the CLI:

```
/plugin marketplace add <GitHubUsername>/<RepoName>
```

Claude will fetch the repo and register the plugin if it finds a valid manifest [9]. You can then use the commands or skills from that plugin in your sessions.

- **Official Registry** – Anthropic maintains an official plugin registry (`claude-plugins-official`) for curated plugins. You can install those by name:

```
/plugin install <plugin-name>@claude-plugins-official
```

(e.g. `/plugin install jira@claude-plugins-official` might install a Jira integration plugin.) This uses a trusted source and is available by default [10].

- **Plugin Scope**: Plugins, once installed, are typically available in all projects for that user (they reside under the user's `~/.claude` config directory). There's not yet a per-project enable/disable mechanism – if installed, their commands/skills load when relevant. To remove a plugin, you can use `/plugin remove`.

- **Current Limitations**: As of Jan 2026, the plugin system does **not** automatically handle certain file types. Notably, `.claude/rules/` files (if your plugin provides custom rules or protocols) are not auto-installed and might require manual copying [65] [66]. This is a known limitation being addressed. Also, there is no GUI "app store" – installation is via CLI as above. All plugins run with the same permissions as Claude Code, so be mindful of security when installing third-party plugins.

MCP Configuration & External Tools

- **What is MCP**: The Model Context Protocol (MCP) allows Claude to interface with external tools, APIs, or data sources in a structured way [67]. An MCP "server" acts as a bridge between Claude and some external system (e.g. a database, an API like Jira or Stripe, or even local hardware). Claude communicates with the MCP server through a standardized JSON interface, treating external functions as additional tools it can use.
- **Adding MCP Servers**: You can add an MCP integration via CLI:

  ```
  claude mcp add <server-name>
  ```

  This will launch a wizard or prompt for details depending on the server. Many community MCP servers are distributed via npm or as executables; the CLI may download or ask you to provide a command. For example, `claude mcp add sentry` might internally run `npx @sentry/mcp-server@latest` to set up Sentry integration [68]. After setup, Claude stores the configuration for that MCP server so it can be used in sessions.
- **MCP Config Files**: Internally, Claude maintains a list of configured MCP servers. **Global (user-wide)** MCPs are stored in the file `~/.claude/claude.json` under an `mcpServers` section [14]. **Project-specific** MCP servers are stored in a `.mcp.json` file in the project root [15]. This allows you to have tools that only appear in one project's context. The `claude mcp add` command by default adds to the current project (creating/ updating `.mcp.json`), but you can specify `--scope user` to add globally (updating your `~/.claude/claude.json`) [13]. Conversely, `--scope project` ensures it's limited to that project.
- **OAuth and Secrets**: Many MCP servers (especially for cloud APIs) require authentication tokens or OAuth setup. Claude Code supports OAuth 2.0 flows in the CLI for certain official servers (it will guide you through a browser auth if needed) [69]. Credentials or tokens are usually stored in the config securely (often encrypted or in OS keychain, depending on server). Enterprise users can also centrally manage credentials via managed-settings if needed.
- **Using MCP Tools**: Once added and running, MCP tools will show up as new tool names in Claude's output. For example, if you add a "Playwright" MCP, Claude might start using a `Playwright` tool to interact with a browser. These interactions will still respect the permission model (you'll be prompted on first use unless you allowed them). You can list all configured servers with `claude mcp list` and remove one with `claude mcp remove <name>` [70]. Logs for MCP servers (especially local ones) are typically stored in `~/Library/Logs/Claude/mcp-*.log` on macOS or similar directories on other OS [71].
- **Scope and Security**: By scoping MCP integrations, you can ensure, for example, that your finance database tools are only active in your finance project, and not available when you're working in an unrelated repository. This scoping (global vs project) prevents accidental use of a tool in the wrong context. *Currently, there isn't an organization-wide MCP registry (each user adds tools individually), but a feature for enterprise MCP registries is under discussion* [72].

“Superuser Pack” Standardization Rules

To establish a consistent Claude Code setup across projects and team members, consider these **standardization practices**:

- **Project Directory Structure**: Every project should have a `.claude/` folder in the root. Inside, include a **project settings** file (`settings.json`) with team-approved defaults (and commit this to VCS) [73]. Also add a blank `settings.local.json` to .gitignore for individuals to use privately (so local tweaks don’t get committed) [22]. This ensures everyone inherits the same baseline config.
- **Permission Baseline**: Define a default set of permission rules that all projects use. For example, put common dangerous actions in deny (like `Bash(rm -rf *)` or `Write(/etc/**)` if relevant) so no one accidentally permits them [40] [74]. Likewise, allow harmless commands like reading docs or running linters to reduce unnecessary prompts [75]. These can live in the project `settings.json`. Establish a convention that deny rules always include certain patterns (like blocking secret files) across all repos for safety.
- **PreToolUse “Safety Protocol”**: Standardize a **“read-me-first”** hook that runs before critical actions. For instance, have a `safety.md` or `PROTOCOL.md` in each repo (outlining dos and don’ts), and add a PreToolUse hook to `cat` that file whenever a high-risk command (like deploying or deleting data) is attempted. This forces Claude to see the safety guidelines in-context. By using the same file name and hook across projects, your team ensures a uniform safety net.
- **Auto-Formatting Hook**: Enable a PostToolUse hook in every project to auto-format code after Claude writes to a file. E.g., in the shared settings.json:

  ```
  "hooks": {
    "PostToolUse": {
      "matcher": { "tool": "Edit" },
      "command": "prettier --write $CLAUDE_FILE_PATH",
      "blocking": false
    }
  }
  ```

  This way, whether working on JS, Python, etc., Claude’s edits are immediately formatted to project standards. Adopting this universally prevents style drift and saves tokens (Claude won’t spend time reformatting). Ensure everyone has the formatter installed in PATH.
- **CLAUDE.md Conventions**: Use a consistent pattern for **CLAUDE.md** across projects. We recommend the “Pointer Pattern” – i.e. CLAUDE.md contains an index of important files or docs rather than full documentation dump [76]. For example, a section listing relevant design docs and their file paths. Also, make it a habit to update CLAUDE.md with any new architecture or API info so that Claude always has an up-to-date knowledge base. This standard practice should be documented so all team members do it, yielding better responses from Claude.
- **Shared “Skills” Library**: Maintain a repository (or folder) of common SKILL files (for company-specific practices or domain knowledge). Developers can symlink or copy these into their `~/.claude/skills/`. Alternatively, create a **plugin** for your org that packages all common skills and commands, so everyone installs the same plugin. This ensures all team members’ Claude instances behave consistently (e.g., everyone has the `SECURITY.md` skill for secure coding checks). Regularly update and communicate changes to these skills.

- **Managed Settings (Enterprise)**: If you have an enterprise setup, use `managed-settings.json` to enforce critical policies: e.g., set `"disableBypassPermissionsMode": "disable"` to prevent anyone from using bypass mode [56], or pre-configure `additionalDirectories` if all projects should allow access to certain shared docs [77]. Also consider using managed settings to set organization-wide defaultModes, sandbox requirements, or telemetry settings (OpenTelemetry can be configured via managed policy as well). This file is distributed by IT and ensures every user's Claude abides by minimum security rules.
- **Version and Keybindings**: Standardize on an **update channel** (e.g., everyone on stable channel vs latest) to avoid version mismatches in features [78]. You can put `"autoUpdatesChannel": "stable"` in managed-settings or ask users to set it in their user config. Additionally, if your team uses specific keybindings (say you remap a certain command), provide a `~/.claude/keybindings.json` template [79] so that everyone can adopt the same shortcuts for muscle memory (this can also be handled via a plugin or simply sharing the file).
- **Parallel Workflow Norms**: If using multi-agent parallel workflows (e.g., multiple git worktrees with CLAUDE_CODE_TASK_LIST_ID), document the standard process so all devs follow it similarly. For example, define that each feature branch uses a separate worktree and an environment variable like `CLAUDE_CODE_TASK_LIST_ID="ProjectName"` to share the task list [80]. By formalizing this, you avoid confusion where one person's Claude isn't seeing tasks created by another's.
- **Backup & Safety Nets**: Encourage use of the community "Safety Net" plugin or a simple backup script (many exist) to periodically save the working directory or protect against Claude mishaps [81]. At minimum, standardize git usage (e.g., always commit or stash before letting Claude refactor large code) and perhaps supply a pre-session checklist (like a git hook or script that runs before `claude` launches, reminding to branch/commit). This isn't a config file per se, but an operational rule to prevent accidental data loss.

By applying these rules across your team's projects, you create a **uniform, predictable Claude Code environment**. This reduces friction (everyone's Claude behaves the same way) and enhances safety (fewer "gotchas" from one person having different settings or forgetting a safeguard).

Remaining Unknowns & Version-Sensitive Items

Even with the above clarifications, a few aspects of Claude Code remain version-dependent or not fully documented as of Jan 2026:

- **TeammateTool (Multi-Agent Swarm)**: Community mentions of a **TeammateTool** for orchestrating agent swarms (multiple Claude instances collaborating) exist, but no official documentation is available [82]. It's unclear if this is an Anthropic-provided feature or a custom community script. Until confirmed, multi-agent "swarm" patterns may require custom coding or third-party tools.
- **Hook Permission Schema**: While we know exit codes control allow/deny, the official docs allude to a possible JSON `permissionDecision: "deny"` response from hooks, but the exact format isn't clearly documented [83]. It's assumed that **exit code 2** is the reliable method (and perhaps in future, hooks could return a structured response to Claude). This might vary slightly by Claude Code version, so when implementing complex hooks, test in your version.
- **Voice Mode Config**: Claude Code's voice input (speech-to-text) can work offline via local Whisper models, but the environment variables to force this have changed between versions. In some versions `STT_BASE_URL` was used; newer versions mention `VOICEMODE_WHISPER_MODEL` to

specify a local model path [84]. Documentation is inconsistent here. If you rely on voice input, check the changelog or experiment to find which variable is effective in your install.

- **Plugin Rule Auto-Import**: As noted, plugin distribution of `.claude/rules/*` files isn't automatic yet [85]. This is a limitation rather than a docs contradiction, but it's something to watch – future updates might enable plugins to include rule files. For now, treat any shared rules as manuals to be added by the user.
- **Parallel Instance Sync**: When running multiple Claude instances (e.g., two worktrees sharing tasks), the task list is shared via a common ID, but real-time synchronization is not instantaneous. It likely polls for updates, meaning there could be a few seconds delay and potential race conditions if two agents edit the same file concurrently [86]. This behavior isn't fully documented; users should avoid simultaneous writes to the same file from multiple agents to be safe.
- **Unity/Godot MCP Servers**: Anthropic has demos of Claude working with game engines (Unity, Godot) via MCP, but there is no official maintained server for these at the moment [87]. Any such integration is community-driven. If your workflow involves game engine automation, be aware you'll likely be using unofficial connectors that might break with updates.
- **Experimental Features**: Some features are version-gated. For example, **Chrome integration** requires Claude Code v2.0.73+ and is only on certain plans [88]. The **Agent SDK** was introduced in mid-2025 (v1.0.23 of the SDK) [88] – ensure your Claude version is up-to-date to use it. Always consult the official changelog [89] for changes like new flags (e.g., a hypothetical `--focus` mode) or deprecations (some older wildcard syntaxes were deprecated in permission rules [90]).

By staying aware of these open questions and checking the latest docs/changelog for your Claude Code version, you can adapt quickly when things evolve. The platform is rapidly improving, and most "unknowns" today likely will be clarified in upcoming releases or documentation updates. In the meantime, the best practice is to test critical configurations in a safe environment and share findings with the Claude Code community.

---

[1] [2] [7] [10] [12] [14] [15] [16] [17] [18] [19] [20] [21] [24] [25] [26] [27] [28] [31] [57] [59] [62] [67] [73] [79] [88] [89] HANDOFF PACKET (Claude Code Mastery) — v1.md
file://file_000000008ed871f89a30363ded6562ee

[3] [4] [29] [30] [36] [37] [65] [66] [84] [85] Prompt B Delta Handoff-Verified Findings-Gemini.pdf
file://file_000000000bdc71f8a9bce58a0e6ada99

[5] [6] [8] [9] [32] [33] [34] [35] [58] [60] [61] [63] [64] [80] [82] [83] [86] [87] PROMPT B — DELTA HANDOFF-Perplexity.md
file://file_00000000d15071f8a8b2422605433b20

[11] [13] Guide for Claude Code | Bito Docs
https://docs.bito.ai/ai-architect/guide-for-claude-code

[22] [23] [41] [42] [48] [49] [50] [51] [52] [53] [54] [55] How to Set Claude Code Permission Mode | ClaudeLog
https://claudelog.com/faqs/how-to-set-claude-code-permission-mode/

[38] [39] [40] [43] [44] [45] [46] [47] [56] [74] [75] [77] [90] Claude Code settings - Claude Code Docs
https://code.claude.com/docs/en/settings

[68] How I use Claude Code - Medium
https://tylerburnam.medium.com/how-i-use-claude-code-c73e5bfcc309

69 Getting Started with Local MCP Servers on Claude Desktop
https://support.claude.com/en/articles/10949351-getting-started-with-local-mcp-servers-on-claude-desktop

70 Claude Code CLI: The Definitive Technical Reference - Blake Crosley
https://blakecrosley.com/en/guides/claude-code

71 Anthropic Connectors Directory FAQ | Claude Help Center
https://support.claude.com/en/articles/11596036-anthropic-connectors-directory-faq

72 Set Up MCP with Claude Code | SailPoint Developer Community
https://developer.sailpoint.com/docs/extensibility/mcp/integrations/claude-code/

76 Claude Code Handoff Packet-Gemini.pdf
file://file_00000000bba071fdb07a161afcff2f5f

78 Set up Claude Code - Claude Code Docs
https://code.claude.com/docs/en/setup

81 Don't let Claude Code wipe your work. I built a "Safety Net" plugin to ...
https://www.reddit.com/r/ClaudeCode/comments/1pvjfau/dont_let_claude_code_wipe_your_work_i_built_a/